

1 Zero-Knowledge Proofs

A *Zero-Knowledge Proof (ZKP)* is a cryptographic protocol that allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that a statement is true without revealing the underlying secret information.

Suppose the prover knows a secret witness w associated with a public statement x . The statement is represented by a relation $R(x, w)$. The prover wants to convince the verifier that it knows a witness satisfying the relation, without revealing the witness itself.

Thus, the basic goal of a ZKP is to

prove knowledge of a secret without revealing the secret.

A ZKP is usually characterized by three fundamental properties.

1. **Completeness:** Completeness guarantees that an honest prover can convince the verifier when the statement is true. More precisely, if $(x, w) \in R$ and both parties follow the protocol, the verifier accepts with overwhelming probability.
2. **Soundness:** Soundness guarantees that a prover who does not possess a valid witness should not be able to convince the verifier that a false statement is true, except with negligible probability.
3. **Zero-Knowledge:** The zero-knowledge property guarantees that the verifier learns nothing about the witness beyond the validity of the statement itself. Therefore, a ZKP simultaneously provides evidence of correctness or knowledge while protecting the underlying secret.

1.1 Sudoku as a ZKP Problem

Consider a prover Alice who claims that she knows a valid solution to a Sudoku puzzle. The Sudoku puzzle is public, while the completed Sudoku grid is Alice's secret witness.

The goal is for Alice to convince Bob that she knows a valid solution without revealing the solution itself.

The public statement is the Sudoku puzzle, and the witness is a completed valid Sudoku grid. The relation $R(\text{Puzzle}, \text{Solution}) = 1$ holds when the proposed solution satisfies all Sudoku constraints.

A simple conceptual construction is based on hiding the solution and allowing the verifier to inspect only randomly selected parts of it.

ZKP of Sudoku Problem The ZKP of Sudoku problem is given as an interactive protocol as follows

1. Alice prepares several identical copies of her completed Sudoku solution.
2. Alice completely hides the entries of each copy using stickers so that Bob cannot see the solution.
3. For each round, Bob randomly selects one hidden copy.
4. Bob randomly selects one of the 27 Sudoku units: one of the 9 rows, one of the 9 columns, or one of the $9 \times 3 \times 3$ boxes.
5. Alice cuts out the selected unit from the chosen copy.
6. Alice separates the nine cells of the selected unit, randomly mixes them, and gives the nine cells to Bob.
7. Bob removes the hiding from the cells and checks that they contain exactly the digits $1, \dots, 9$.
8. The above procedure is *repeated* independently for multiple copies, with a fresh random challenge in each round.

If Alice has a valid Sudoku solution, every challenge is answered successfully. If Alice does not have a valid Sudoku solution, Bob has a non-negligible probability of selecting an invalid row, column, or 3×3 box and detecting the cheating.

2 ZKP for Machine Learning as a Service

Consider a machine-learning service operated by a server $\mathcal{S}$. A client sends an input x , and the server evaluates a model f to obtain $y = f(x)$. Suppose the client does not need to learn the exact value of y . Instead, the client only wants to know whether the result belongs to a specified range $[L, U]$. The desired result is therefore the predicate $y \in [L, U]$.

The server should convince the client that this condition is correct without revealing unnecessary information about the computation. This can be formulated as a zero-knowledge proof in which the server proves the validity of a statement involving its hidden computation.

2.1 Public Information and Private Information

Before constructing a ZKP, it is necessary to determine which information is public and which information must remain private.

Depending on the application, the public information may include the model description or a commitment to the model, a public input, the required output range $[L, U]$, and the security parameters.

Private information may include the input x , model parameters, the exact output y , and intermediate computation values.

Thus, the prover's secret information can be represented as a witness w , while the verifier receives a public statement x_{pub} .

The ZKP then establishes that there exists a valid witness satisfying the required relation.

2.2 The Importance of the Data and Query

The required ZKP construction depends strongly on what information must remain private.

For example, a system may require the input to be private while the model is public. Another system may require the model to remain private while the input is public. In a stronger setting, both the input and the model may be private.

Similarly, the exact output may need to remain hidden while only a predicate about the output is revealed.

For the range-query setting, the server computes $y = f(x)$, but the verifier should learn only whether $y \in [L, U]$.

Thus, the ZKP statement must capture the required computation and the desired predicate without unnecessarily exposing the underlying witness.

The important principle is:

The statement to be proved determines the appropriate ZKP construction.

3 Examples

3.1 Proving Knowledge of a Secret Key

Consider a public-key encryption scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ with key generation

$$(pk, sk) \leftarrow \text{KeyGen}(1^\lambda).$$

Suppose a prover claims to know the secret key sk corresponding to the public key pk . The goal is to prove knowledge of sk without revealing sk itself.

This type of proof is mainly used when obtaining a certificate from a Certificate Authority (CA). For example, when a user requests a digital certificate for a public key pk , the CA needs to verify that the requester actually possesses the corresponding private key sk . The private key should never be sent to the CA.

Interactive Challenge-Response Protocol Assume an public-key encryption scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ with public key pk . The protocol proceeds as follows:

1. The verifier $\mathcal{V}$ chooses a fresh random value r .
2. The verifier computes the ciphertext

$$c = \text{Enc}(pk, r).$$

3. The verifier sends c to the prover $\mathcal{P}$.

4. The prover uses the secret key sk to decrypt the ciphertext and obtains

$$r' = \text{Dec}(sk, c).$$

5. The prover sends r' to the verifier.

6. The verifier accepts if and only if

$$r' = r.$$

The value r is freshly generated by the verifier and is unknown to the prover before receiving the ciphertext. Therefore, a prover that can consistently produce the correct response demonstrates possession of the corresponding secret key.

At no point does the prover reveal sk to the verifier. The protocol therefore provides a way to demonstrate knowledge of the secret key without directly disclosing it.

3.2 Example: ElGamal and the Schnorr Protocol

A more concrete example of a proof of knowledge is obtained from the ElGamal public-key system.

Let $\mathbb{G}$ be a cyclic group of prime order q with generator g . The secret key is a random value $x \in \mathbb{Z}_q$, and the corresponding public key is

$$y = g^x.$$

The question is whether a prover can demonstrate knowledge of x without revealing x .

Schnorr Identification Protocol The Schnorr protocol provides an interactive proof of knowledge of the discrete logarithm x .

The prover first chooses a random value $r \in \mathbb{Z}_q$ and computes the commitment

$$a = g^r.$$

The prover sends a to the verifier.

The verifier then chooses a fresh random challenge $c \in \mathbb{Z}_q$ and sends it to the prover.

Using the secret x , the prover computes

$$z = r + cx \pmod{q}$$

and sends z to the verifier.

The verifier accepts if

$$g^z = a y^c.$$

Correctness of Schnorr For an honest prover,

$$g^z = g^{r+cx} = g^r (g^x)^c = a y^c.$$

Therefore, the verification equation holds.

The secret x is never revealed. The verifier sees only the commitment a , the challenge c , and the response z .

The underlying relation being proved is

$$R(y, x) = 1 \iff y = g^x.$$

Thus, Schnorr provides a proof of knowledge of x satisfying the public relation $y = g^x$.

Underlying Security Assumption The security of the classical Schnorr protocol relies on the difficulty of the discrete logarithm problem in the underlying group.

Given g and $y = g^x$, an adversary should not be able to efficiently recover x .

This illustrates an important distinction from information-theoretic constructions: the security of Schnorr depends on a computational hardness assumption.

4 From Interactive to Non-Interactive ZKP

The Schnorr protocol is interactive because the verifier must generate and send the challenge after receiving the prover's commitment. The communication pattern is $\mathcal{P} \rightarrow \mathcal{V} \rightarrow \mathcal{P} \rightarrow \mathcal{V}$.

In many applications, it is desirable to remove this interaction. The prover should be able to generate a proof that can later be transmitted to and verified by another party.

A common approach is to derive the challenge from a cryptographic hash function.

4.1 Fiat-Shamir Transformation

The Fiat-Shamir transformation replaces the verifier's random challenge with a hash-derived challenge.

In the Schnorr protocol, the prover computes the commitment $a = g^r$. Instead of receiving a random challenge from the verifier, the prover computes

$$c = H(y, a).$$

The prover then computes

$$z = r + cx \pmod{q}.$$

The resulting proof can be represented by the pair (a, z) .

To verify the proof, the verifier recomputes $c = H(y, a)$ and checks whether

$$g^z = a y^c.$$

Thus, the verifier no longer needs to interact with the prover to generate the challenge.

The Fiat-Shamir transformation therefore converts an interactive challenge-response protocol into a non-interactive proof by deriving the challenge from a hash function.

4.2 Why Non-Interactive Proofs?

Non-interactive proofs are useful when a proof needs to be generated once and verified later. The proof may be transmitted to another party, verified by multiple parties, or incorporated into a larger cryptographic system.

A non-interactive proof can be viewed abstractly as

$$\pi = \text{Prove}(x, w),$$

where x is the public statement and w is the witness. Verification is performed using

$$\text{Verify}(x, \pi) \in \{0, 1\}.$$

The important transition is from an interactive challenge-response protocol to a transferable proof.

5 Towards Range Proofs

Return to the machine-learning example in which the server computes $y = f(x)$.

Suppose the server wants to prove that

$$L \leq y \leq U$$

without revealing the exact value of y .

The desired proof should establish that there exists a valid hidden computation whose output lies in the required range. The verifier should learn that the range condition holds without learning the hidden value itself.

This leads to the notion of a *range proof*.

A range proof is therefore a zero-knowledge proof that a hidden value belongs to a specified public range.

In the machine-learning setting, the server can use such a proof to demonstrate that the result of a private computation satisfies a required condition while keeping the exact result hidden.